

Algorithmique Avancée

Examen Réparti 2

Les seuls documents autorisés sont les polys de cours, ainsi que la copie personnelle. Le barème donné est indicatif. Les réponses aux diverses ne seront considérées que lorsqu'elles seront justifiées.

Exercice 1 : QCM [4 points]

Dans ce QCM, pour chaque question vous devez donner 1 seule réponse et expliquer votre choix par une ligne de texte ou une figure. Le barème est le suivant : **réponse correcte et correctement argumentée : 1 point. Réponse incorrecte ou correcte mais mal argumentée : 0 point.** Il n'y a pas de points négatifs.

Question 1 Soit un polygone P dont on sait que l'enveloppe convexe E contient ℓ sommets.

- A. Le polygone P a 2ℓ sommets.
- B. Tout point à l'intérieur de l'enveloppe convexe E est un sommet à l'intérieur du polygone P .
- C. Tout sommet de l'enveloppe convexe E est un sommet du polygone P .

Question 2 Le calcul du coût amorti permet d'analyser un algorithme en moyenne ?

- A. Vrai.
- B. Faux.

Question 3 Il existe un algorithme qui permet de trier un tableau d'entiers en temps linéaire (par rapport à la taille du tableau), si les entiers sont compris entre 1 et 1000000.

- A. Vrai.
- B. Faux.

Question 4 Si je prouve que la complexité dans le pire des cas d'un algorithme est en $\Theta(n^2)$, est-il possible qu'il soit en $O(n)$ sur certaines données ?

- A. Vrai.
- B. Faux.

Exercice 2 : Arbres quasi-équilibrés [6 points]

On rappelle qu'un arbre binaire strict est soit réduit à une feuille (1 nœud externe) ou possède une racine (un nœud interne) et deux fils qui sont eux-même des arbres binaires.

Dans cet exercice, tous les arbres sont binaires stricts. La hauteur h d'un arbre T est telle que : si T est une feuille alors $h(T) = 0$, et sinon $h(T) = 1 + \max\{h(T1), h(T2)\}$, où $T1$ et $T2$ sont les sous-arbres à la racine de T .

Soit $\mathcal{P}_c$ la propriété suivante définie sur les arbres : un arbre T vérifie la propriété $\mathcal{P}_c$ si et seulement si il existe une constante c telle que pour tout nœud interne ν de T , les hauteurs des sous-arbres de ν diffèrent au plus de c .

Question 1 Caractériser la famille des arbres qui vérifient $\mathcal{P}_0$ et celle des arbres qui vérifient $\mathcal{P}_1$.

Question 2 Donner un exemple d'arbre vérifiant $\mathcal{P}_2$ mais pas $\mathcal{P}_1$ et un exemple d'arbre ne vérifiant pas $\mathcal{P}_2$.

Question 3 Montrer que pour tout $c \in \mathbb{N} \setminus \{0\}$, il existe $\alpha_0 \in]0, 1]$, tel que pour tout $\alpha \in]0, \alpha_0]$ on ait $\alpha(1 + \alpha)^c \leq 1$.

Question 4 Montrer, par récurrence sur la hauteur, que pour tout arbre de taille n et de hauteur h , qui vérifie $\mathcal{P}_c$, il existe $0 < \alpha \leq 1$ tel qu'on ait $n \geq (1 + \alpha)^h - 1$.

Question 5 En déduire que tout arbre, qui vérifie la propriété $\mathcal{P}_c$, a une hauteur en $O(\log n)$.

Exercice 3 : Tri dans des structures classiques [6 points]

Question 1 Soit une file binomiale contenant n éléments. En utilisant les primitives sur les files binomiales, donner un algorithme qui retourne la liste triée des éléments. Donner la complexité de cet algorithme dans le pire des cas.

Question 2 Soit un arbre bicolore contenant n éléments. En utilisant les primitives sur les arbres binaires, donner un algorithme qui retourne la liste triée des éléments. Donner la complexité de cet algorithme dans le pire des cas.

Question 3 Soit un trie hybride contenant n nœuds internes. En utilisant les primitives sur les arbres ternaires donner un algorithme qui retourne la liste triée des mots représentés par le trie. Donner la complexité de cet algorithme dans le pire des cas.

Exercice 4 : Enveloppe convexe [6 points]

D'un point de vue de structure de données en 2D, un *polygone* est une liste ordonnée de points dans le plan. Avec cette structure de données, on appelle *côté* du polygone tout segment reliant deux points consécutifs dans la liste définissant le polygone. Un polygone est *simple* si la seule intersection possible entre deux côtés quelconques du polygone sont des extrémités de ces deux côtés.

Par abus de langage, on confond le polygone et la partie du plan situé à l'intérieur des segments délimitant le polygone. Un polygone est *convexe* s'il est simple et s'il contient tout segment reliant deux points à l'intérieur du polygone. L'*enveloppe convexe* d'un objet géométrique (liste de points, liste de segments, polygone, ...) est le plus petit polygone convexe contenant cet objet. Dans les questions suivantes, les seuls langages de programmation autorisés sont : java, python et C.

Question 1 de cours :

Donner le code d'une fonction qui prend en entrée un polygone simple comprenant n points et qui retourne en temps $O(n)$ l'enveloppe convexe de P .

Question 2 Diviser-pour-régner en temps $O(n \log n)$:

- Donner le code d'une fonction qui prend en entrée deux polygones convexes P_1, P_2 comprenant au total n points et qui retourne en temps $O(n)$ l'enveloppe convexe de $P_1 \cup P_2$.
- Utiliser la précédente implémentation et donner le code d'une fonction récursive qui prend en entrée une liste quelconque de n points et qui retourne en temps $O(n \log n)$ l'enveloppe convexe de ces points.

N.B. : il est impératif que la fonction soit récursive, et que le paradigme algorithmique soit de type diviser-pour-régner.

Question 3 Optimalité :

Nous supposons dans cette question avoir accès à une fonction `enveloppeConvexe` prenant en entrée n points dans le plan et retournant en temps τ l'enveloppe convexe de ces points.

Donner le code d'une fonction qui prend en entrée une liste quelconque de n entiers et qui retourne en temps $O(n + \tau)$ ces entiers triés par ordre croissant.

En utilisant uniquement la notation $\Theta()$, donner la complexité au pire cas d'un algorithme calculant l'enveloppe convexe d'un ensemble de n points dans le plan.